

Appendix S2. Data simulation and analysis for the FP-FN abundance model

Ismael V. Brack Denis Valle

April 26, 2026

Table of contents

Required Packages	1
1. Data simulation	2
Simulation settings	2
Simulate abundance in sites	2
Simulate data collection and automated detections	3
Simulate object-level data	4
2. Data analysis	5
Specify model for JAGS	6
Organize data for JAGS	7
Fit the FP-FN abundance model	8

This Appendix is a supporting document for the manuscript “*Estimating wildlife occupancy and abundance using automated object detections in images*” (Brack et al., 2026). First, we show the data simulation process to generate the automated count data containing true-positive (TP) and false-positive (FP) detections and their associated confidence scores, as well as the independent human detections in the verification subset. Then, we fit our FP-FN abundance model to the simulated data. The model relies on a double verification procedure that is applied to a subset of images: i) algorithm detections in this subset are verified and classified into TP and FPs; and ii) an independent person (with no access to the algorithm detections) manually reviews this same subset and mark every individual detected. The abundance of each site is then estimated by adjusting the TP counts for the detection probabilities estimated for each site using a canonical formula of population size.

Required Packages

We will need the following packages to run this code:

```
library(jagsUI)
library(ggplot2)
library(MCMCvis)
```

Note that you also need to have installed JAGS software in your PC. You can download it [here](#).

1. Data simulation

In this example, we will simulate 20 sites with a site abundance varying between 20 and 200 individuals. While we keep human and algorithm detection probabilities as constant, we simulate the algorithm precision varying between sites. We simulate two types of data: verified and unverified counts per site, and object-level detections with corresponding classes and confidence scores.

Simulation settings

First, we define the true values for the parameters and define the simulation settings for the sampling design:

```
nsites = 20 # number of sites

abund.rng = c(20,200) # abundance range

pa = 0.8 # algorithm detection probability (recall)
ph = 0.7 # human detection probability (independent manual review)

precision = runif(nsites,0.2,0.8) # algorithm precision: proportion of TPs

prop2obs = 1/10 # proportion of the drone images with double verification

tp.mu = qlogis(.75) # mean score for TPs
fp.mu = qlogis(0.5) # mean score for FPs

sd.scr = 0.5 # sd for scores
```

Simulate abundance in sites

Next, we simulate a total abundance for each site and split this abundance into the two components that will receive the different subset processing procedures.

```
## Simulate local abundance in sites
N = round(runif(nsites, abund.rng[1],abund.rng[2]))
N1 = round(prop2obs * N) # true abundance in the verification subset
N2 = N - N1 # true abundance in the remaining site
```

	N	N1	N2
	103	10	93
	37	4	33
	161	16	145
	189	19	170
	35	4	31
	23	2	21
	23	2	21
	46	5	41

N	N1	N2
58	6	52
88	9	79
44	4	40
120	12	108
160	16	144
140	14	126
54	5	49
68	7	61
128	13	115
67	7	60
173	17	156
50	5	45

Simulate data collection and automated detections

The dataset is simulated with two components: a subset of flights with double verification that contains the multinomial TP counts and FP detections, while the remaining images are composed only of unverified algorithm counts (TP+FP). We simulate the human and algorithm detections as independent processes and we assume that human reviewers do not have FP errors (only FN). The unverified counts are the sum of TP and FP detections by the algorithm.

```
## Simulate data collection

# Multinomial TP counts from N1
Y <- array(dim=c(nsites,4),
           dimnames=list(1:nsites,c("11","10","01","00")))

for(s in 1:nsites){
  # human detections for each individual in N1
  obs.human <- rbinom(N1[s],1,ph)
  # algorithm detections for each individual in N1
  tp.algo <- rbinom(N1[s],1,pa)
  # combine detections into counts
  temp.c <- table(factor(paste0(obs.human,tp.algo),
                        levels=colnames(Y))
)

  Y[s,] <- temp.c
} #s

# Algorithm true positives
TP1 <- rowSums(Y[,c("11","01")]) # verified subset
TP2 <- rbinom(nsites,N2,pa) # unverified subset

# Algorithm false positives
FP1 <- rpois(nsites, round(TP1*(1-precision) / precision)) # verified subset
```

```

FP2 <- rpois(nsites, round(TP2*(1-precision) / precision)) # unverified subset

# Resulting data
counts <- cbind(Y[, -4], FP1, unverified= TP2 + FP2)

```

Note above that we convert the algorithm precision into an expected rate to simulate the number of FPs using a Poisson distribution.

We show below the resulting count object containing the multinomial counts in the double verified dataset, where 11: detected by both the human and algorithm; 10: detected only by the human; 01: detected only by the algorithm; FP1: false positives; and the unverified counts for the remaining images.

11	10	01	FP1	unverified
4	1	3	6	117
2	1	1	5	59
10	3	2	8	172
12	2	3	38	398
2	1	1	10	102
1	1	0	4	77
2	0	0	5	55
2	2	1	1	67
2	0	3	5	102
5	0	3	3	99
1	1	2	0	42
5	1	4	4	150
13	1	2	37	348
7	1	5	10	203
4	1	0	0	80
5	1	1	1	79
7	4	2	6	144
4	0	3	9	94
9	3	4	38	556
4	1	0	5	67

Simulate object-level data

Now, we use the counts corresponding only to the algorithm detections (excluding human detections) to extract the class of each detected object and simulate their confidence scores. Object scores are simulated according to its true class (TP or FP) distribution. The object class is only observed for those detections that are on the verified subset.

```

## Simulate object-level data
obj.data <- list()

oo=0
for(i in 1:nsites){
  oo=oo+1
  # True/False detection

```

```

tab <- data.frame(
  # true partially observed class
  class=c(rep(1,TP1[i] + TP2[i]) ,rep(0,FP1[i] + FP2[i])),
  # observed classes
  obs.class=c(rep(c(1,NA), c(TP1[i],TP2[i])),
               rep(c(0,NA), c(FP1[i],FP2[i]))),
  score=NA,
  site=i
)
# Scores for each detected object
# TP scores
tab[which(tab$class==1),"score"] <- rnorm(TP1[i] + TP2[i], tp.mu, sd.scr)
# FP scores
tab[which(tab$class==0),"score"] <- rnorm(FP1[i] + FP2[i], fp.mu, sd.scr)

obj.data[[oo]] <- tab
}#i

obj.data <- do.call(rbind, obj.data) # transform to data frame

```

This is how the object-level data looks like:

class	obs.class	score	site
1	NA	0.9894271	3
1	NA	0.9922767	1
1	NA	0.6731741	15
0	NA	1.0622029	8
1	NA	1.1310851	10
0	NA	-0.0526386	17
0	NA	0.1848703	3
1	NA	1.6825712	19
0	NA	0.1695311	10
0	NA	0.0994698	19
0	0	-0.3571320	19
0	NA	0.7029615	4
0	NA	-0.3945486	6
1	1	0.9607191	12
0	NA	0.0356802	19

2. Data analysis

For analyzing the data, we will use the two objects shown above: the count data (containing the multinomial observations and the unverified counts) and the object-level data.

Specify model for JAGS

We specify the JAGS model using the code below. We estimate separately the latent abundance at each subset of each site, and then sum these abundances to obtain the total abundance on the site. Note that we simulated the precision as an independent value for each site (from a uniform distribution) but we model it using a site-level random intercept (from a normal distribution). Interestingly, the model is robust to this difference.

```
abund.model <- "
model{

# Priors -----

pa ~ dunif(0,1) # algorithm detection prob.
ph ~ dunif(0,1) # human detection prob.

# Algorithm precision (TP prop.) - random effects
mu.prec ~ dnorm(0, 0.01)
sd.prec ~ dunif(0,5)
tau.prec <- 1 / sd.prec^2

# Scores
mu.scr[1] ~ dnorm(0, 0.01) # mean score values for class=0
mu.dif ~ dunif(0,5)      # impose restriction: mu2 >= mu1
mu.scr[2] <- mu.scr[1] + mu.dif # mean score value for class=1

sd.scr[1] ~ dunif(0, 10) # std.dev. for FP scores
sd.scr[2] ~ dunif(0, 10) # std.dev. for TP scores
tau.scr[1] <- 1 / sd.scr[1]^2
tau.scr[2] <- 1 / sd.scr[2]^2

# Likelihood -----

for(s in 1:nsites){
  # Define multinomial cell probabilities
  pi[s,1] <- pa*ph      # 11
  pi[s,2] <- (1-pa)*ph  # 10
  pi[s,3] <- pa*(1-ph)  # 01

  # Conditional multinomial probs.
  for(k in 1:3){
    pic[s,k] <- pi[s,k] / sum(pi[s,])
  }

  pcap[s] <- 1 - (1-pa)*(1-ph) # overall detection prob.

  Y[s,] ~ dmulti(pic[s,], ndet1[s]) # observarion of encounter histories

  # Algorithm precision random effects
  alpha[s] ~ dnorm(mu.prec,tau.prec)
```

```

logit(precision[s]) <- alpha[s]

# True positives in the unverified counts
TP2[s] ~ dbin(precision[s], ndet2[s])

## Derived abundance estimates for each subet
N1[s] <- ndet1[s] / pcap[s]
N2[s] <- TP2[s] / pa

# Total abundance per site
N[s] <- N1[s] + N2[s]

} #s

## Scores
for(i in 1:nobj){
  class[i] ~ dbern(precision[site[i]]) # true latent class (TP of FP)

  score[i] ~ dnorm(mu.scr[(class[i]+1)], tau.scr[(class[i]+1)]) # object-level scores
} #i
} "

# Save model mode as a temporary .txt file
temp.mod2 <- tempfile(fileext = ".txt")
writeLines(abund.model, temp.mod2)

```

Organize data for JAGS

We start by bundling the data required to run the JAGS model:

```

# Bundle data
dat <- list(
  nsites=nsites,
  # double verified data
  Y=counts[,1:3],
  ndet1=rowSums(counts[,1:3]),
  # unverified data
  ndet2=counts[, "unverified"],
  # object data
  class=obj.data$obs.class,
  score=obj.data$score,
  site=obj.data$site,
  nobj=nrow(obj.data)
)

```

Initial values for the MCMC algorithm:

To avoid issues with initial values, we define TP for all the unobserved object classes:

```
# Initial values
inits <- function() list(
  class=ifelse(is.na(obj.data$obs.class),1,NA)
)
```

Next, we define the parameters that we want to be monitored in MCMC and define the MCMC settings.

```
# Parameters monitored
params <- c("pa","ph","mu.prec","sd.prec","mu.scr","precision","N","N1","N2")

# MCMC settings
ni <- 20000 # number of iterations
nt <- 1 # thinning rate
nb <- 5000 # burn-in
nc <- 3 # number of chains
```

Fit the FP-FN abundance model

Finally, we run the abundance model (note it can take a few minutes to run):

```
out <- jags(dat, inits, params,
  model.file=temp.mod2,
  n.chains=nc,n.thin=nt, n.iter=ni, n.burnin=nb,
  parallel=T)
```

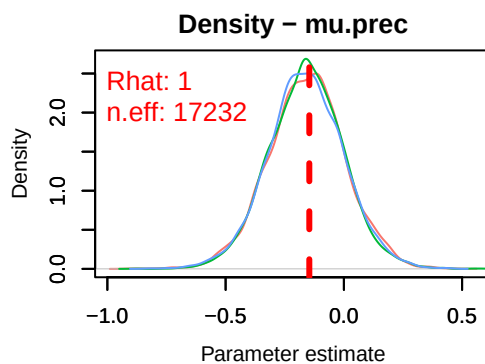
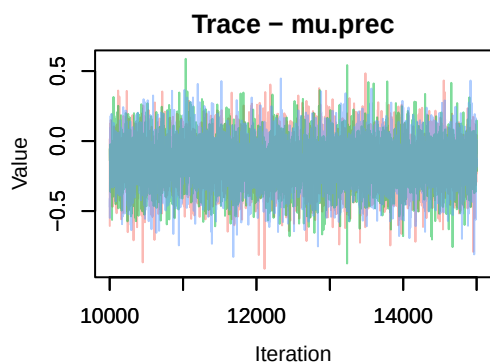
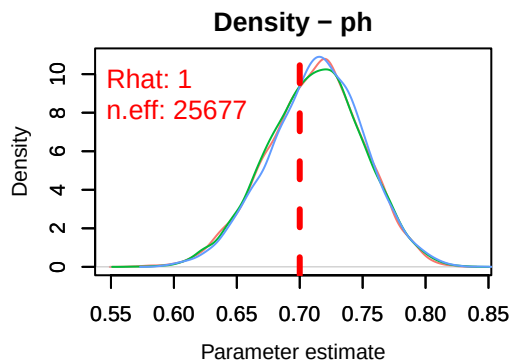
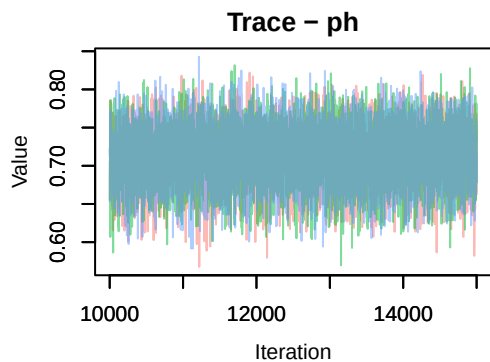
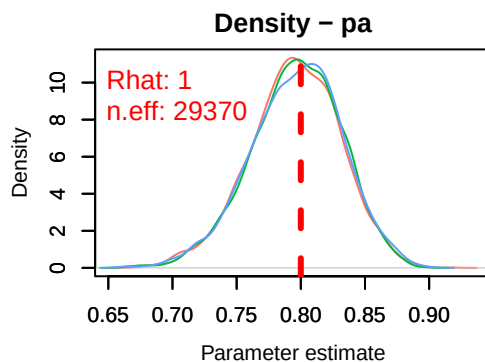
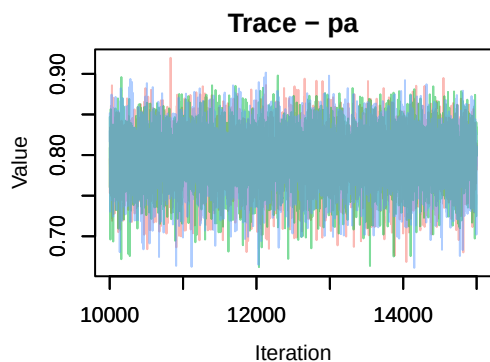
Let's see the summarized posterior samples to examine the model results:

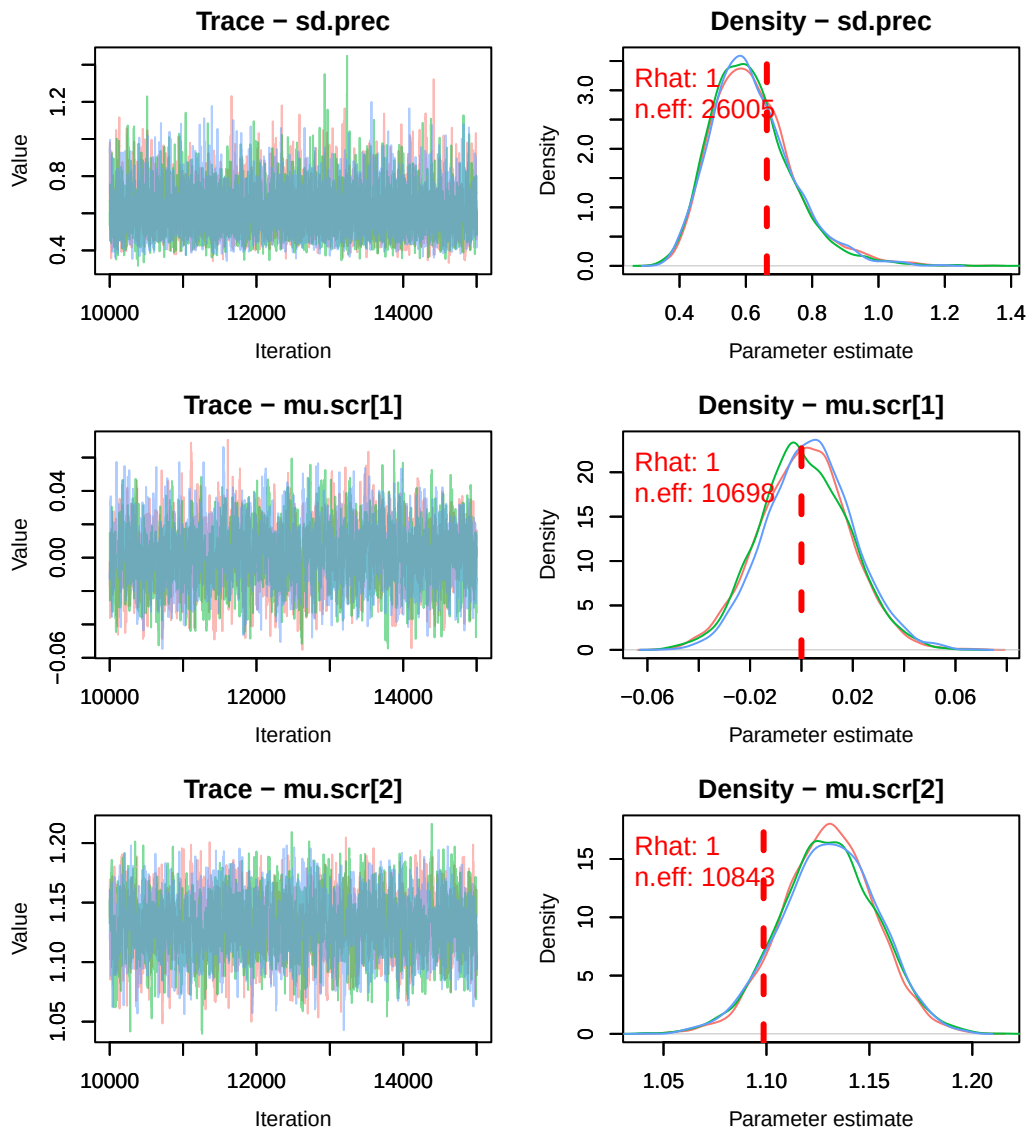
```
print(round(out$summary[1:6,-11],3))
```

	mean	sd	2.5%	25%	50%	75%	97.5%	Rhat	n.eff	overlap0
pa	0.796	0.035	0.723	0.774	0.798	0.821	0.861	1	29370	0
ph	0.712	0.038	0.636	0.687	0.714	0.738	0.783	1	25677	0
mu.prec	-0.155	0.160	-0.471	-0.260	-0.156	-0.050	0.166	1	17232	1
sd.prec	0.620	0.124	0.421	0.532	0.605	0.689	0.912	1	26005	0
mu.scr[1]	0.001	0.018	-0.033	-0.011	0.001	0.013	0.036	1	10698	1
mu.scr[2]	1.130	0.024	1.083	1.114	1.130	1.146	1.176	1	10843	0

We can also see the traceplots and posterior distributions for the monitored parameters. Note that each chain is represented by one color and the vertical red line represent the true parameter value used to simulate data.

```
# traceplots..
MCMCtrace(out,ind=T,pdf=F,
  Rhat=T,n.eff=T,
  params=c("pa","ph","mu.prec","sd.prec","mu.scr"),
  gvals=c(pa,ph,
    mean(qlogis(precision)),sd(qlogis(precision)),
    fp.mu,tp.mu) )
```



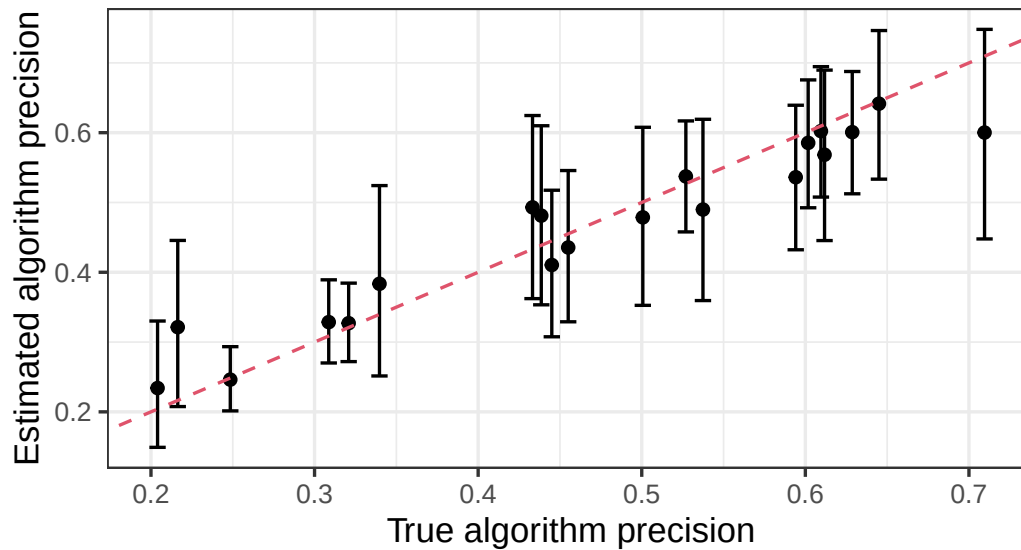
Parameter estimates look good! Estimated values are close to the true values used to simulate the data. It is important to check MCMC convergence in the results (e.g., `Rhat<1.1`).

Now, we take a look at the precision and abundance estimates for each site. Let's organize the results for both estimates and compare with the true values used to simulate data:

```
prec.resu <- data.frame(
  true = precision,
  mean = out$mean$precision,
  lcl = out$q2.5$precision,
  ucl = out$q97.5$precision
)

ggplot(data=prec.resu, aes(x=true,y=mean,ymin=lcl,ymax=ucl)) +
  geom_errorbar(width=0.01) +
  geom_point() +
  labs(y="Estimated algorithm precision",x="True algorithm precision") +
```

```
geom_abline(intercept=0,slope=1,col=2,linetype="dashed") +
theme_bw(base_size=13)
```

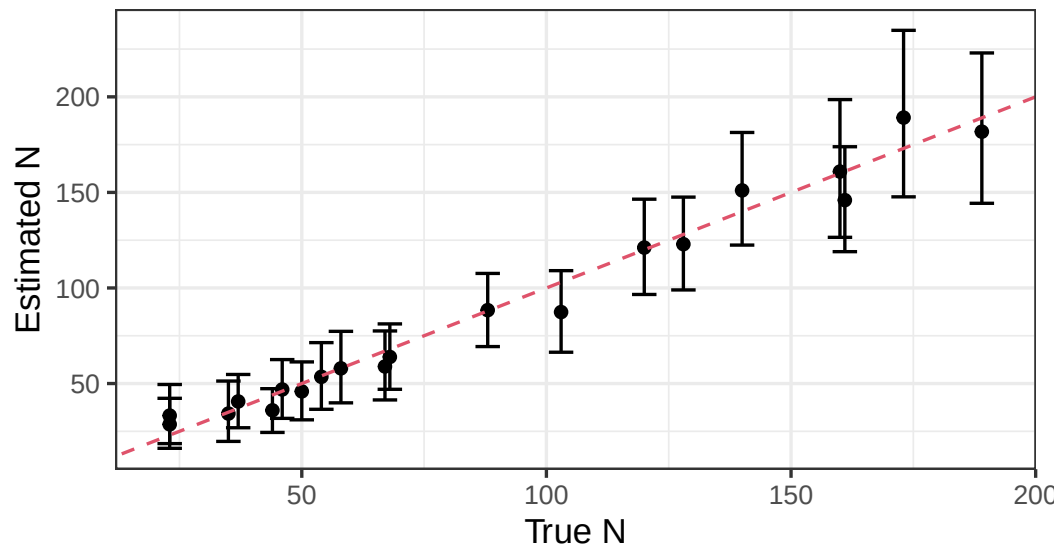


We also plot the 1:1 relationship line in red to evaluate the results.

We do the same for the abundance estimates:

```
N.resu <- data.frame(
  true = N,
  mean = out$mean$N,
  lcl = out$q2.5$N,
  ucl = out$q97.5$N
)

ggplot(data=N.resu, aes(x=true,y=mean,ymin=lcl,ymax=ucl)) +
  geom_errorbar(width=5) +
  geom_point() +
  labs(y="Estimated N",x="True N") +
  geom_abline(intercept=0,slope=1,col=2,linetype="dashed") +
  theme_bw(base_size=13)
```



Both figures are showing that the estimated values of precision and abundance are capturing the true values used in simulation!